

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 08 -

PROJETO DE COMPARAÇÃO DE MODELOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	8
MERCADO, CASES E TENDÊNCIAS	19
O QUE VOCÊ VIU NESTA AULA?	25
REFERÊNCIAS.....	27

EMSE

O QUE VEM POR AÍ?

Ao desenvolver um projeto de Machine Learning, é tentador escolher o modelo com melhor acurácia no conjunto de treino e seguir em frente. Contudo, comparar modelos de forma rigorosa vai muito além de olhar um número isolado: envolve avaliar desempenho em dados nunca vistos, considerar custos computacionais e garantir que os resultados sejam reproduzíveis.

Por exemplo, imagine dois modelos que resolvem a mesma tarefa: um modelo simples e rápido e um mais complexo e pesado (consome muita GPU). Se adotarmos o modelo pesado sem análise profunda apenas porque teve acurácia ligeiramente maior em um teste inicial, podemos enfrentar “períodos ociosos” equivalentes aos pods subutilizados: eventualmente o modelo complexo ficará consumindo recursos de GPU sem trazer ganho significativo, aumentando custos.

Foi diante dessas questões que surgiram metodologias modernas de orquestração de experimentos de ML. Em vez de se basear apenas em uma métrica estática, essas abordagens monitoram múltiplos “gatilhos” no projeto de modelos – desempenho em diversos subconjuntos de validação, consumo de memória/CPU, tempo de inferência etc. – e permitem decisões dinâmicas sobre qual modelo utilizar ou quando treinar um novo. Isso viabiliza estratégias como alternar para um modelo mais simples em períodos de menor demanda, ou ativar modelos complexos sob demanda quando um padrão de dados mais desafiador é detectado.

Outra vantagem dessa visão orientada a projeto é integrar o processo de comparação de modelos ao ciclo de vida de Machine Learning de forma dinâmica e iterativa. Com isso, chegamos a um cenário em que modelos ficam “totalmente inativos quando não há necessidade” (por exemplo, não gastamos horas de GPU comparando modelos irrelevantes) e só entram em ação quando realmente há demanda ou potencial de ganho. Em suma, nesta aula veremos como conduzir projetos de comparação de modelos de forma inteligente – aprendendo a ligar e desligar recursos computacionais conforme a necessidade, avaliar resultados com rigor estatístico e econômico e entregar soluções de IA melhores, no momento certo, pelo motivo certo.

HANDS ON

Chegou a hora de colocar em prática o que falamos. Vamos supor um cenário simples: você possui um conjunto de dados e quer decidir entre dois algoritmos de classificação – Modelo A: uma regressão logística (mais simples) e Modelo B: uma pequena rede neural (mais complexa). Nosso objetivo é comparar os dois de maneira justa, medindo desempenho e também observando o custo computacional (mesmo que básico, como tempo de treinamento). Em projetos reais, faríamos validação cruzada extensa, mas aqui, para simplificar a ilustração, usaremos uma única divisão treino/teste mantendo controle de aleatoriedade.

O plano: definir um experimento reproduzível onde ambos os modelos serão treinados e avaliados nas mesmas condições. Para isso, geramos um conjunto de dados sintético não-linear (onde um modelo linear deve ter dificuldade) e separamos em treino e teste. Fixamos uma seed global para que qualquer aleatoriedade – geração dos dados, inicialização dos pesos, ordem de amostragem – seja igual para os dois modelos, garantindo uma comparação imparcial. Treinaremos cada modelo e registraremos sua acurácia no conjunto de teste, o tempo gasto para treinar e até o número de parâmetros (como proxy de complexidade). Esperamos que o Modelo B obtenha acurácia mais alta, mas vamos ver se essa melhoria compensa o custo extra.

No código a seguir, implementamos esse experimento em Python. Usamos a biblioteca PyTorch para definir os modelos de forma bem direta (um com apenas uma camada linear, outro com uma camada oculta) e realizar o treinamento por algumas épocas.

```
1     import torch
2     import torch.nn as nn
3     import torch.optim as optim
4     import numpy as np
5
6     # 1. Reprodutibilidade: fixa seed para numpy e
PyTorch
7     np.random.seed(42)
8     torch.manual_seed(42)
9
10    # 2. Geração de dados sintéticos (não linearmente
separáveis) e divisão treino/teste
```

```
11      # Aqui usamos um dataset "moons" para ter classes
em meia-lua
12      from sklearn.datasets import make_moons
13      from sklearn.model_selection import
train_test_split
14      X, y = make_moons(n_samples=300, noise=0.10,
random_state=42)
15      X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.33, random_state=42)
16      # Converte para tensores do PyTorch
17      X_train = torch.tensor(X_train,
dtype=torch.float32)
18      y_train = torch.tensor(y_train,
dtype=torch.float32).view(-1, 1)
19      X_test = torch.tensor(X_test, dtype=torch.float32)
20      y_test = torch.tensor(y_test,
dtype=torch.float32).view(-1, 1)
21
22      # 3. Definição dos dois modelos a comparar:
23      modelA = nn.Sequential(
24          nn.Linear(2, 1),
25          nn.Sigmoid()
26      )
27      modelB = nn.Sequential(
28          nn.Linear(2, 10),
29          nn.ReLU(),
30          nn.Linear(10, 1),
31          nn.Sigmoid()
32      )
33
34      # Hiperparâmetros de treinamento
35      num_epochs = 1000
36      lr = 0.1
37
38      # 4. Função auxiliar para treinar e avaliar um
modelo
39      def treinar_e_avaliar(model, nome):
40          criterion = nn.BCELoss() # perda para
classificação binária
41          optimizer = optim.SGD(model.parameters(),
lr=lr)
42          # Treinamento
43          inicio = torch.cuda.Event(enable_timing=True);
fim = torch.cuda.Event(enable_timing=True)
44          inicio.record() # marca tempo inicial (usando
eventos CUDA para alta precisão)
45          for epoch in range(num_epochs):
46              optimizer.zero_grad()
47              outputs = model(X_train)
48              loss = criterion(outputs, y_train)
49              loss.backward()
```

```

50         optimizer.step()
51         fim.record(); torch.cuda.synchronize() # marca
tempo final e sincroniza
52         tempo_ms = inicio.elapsed_time(fim) # tempo em
milissegundos
53         # Avaliação
54         with torch.no_grad(): # desliga gradiente para
teste
55             preds = model(X_test)
56             preds_classes = (preds >= 0.5).float()
57             acertos = (preds_classes ==
y_test).sum().item()
58             acc = acertos / y_test.shape[0]
59             # Coleta métricas do modelo
60             total_params = sum(p.numel() for p in
model.parameters())
61             print(f"{nome}: Acurácia = {acc*100:.2f}% |
Parâmetros = {total_params} | Tempo treino =
{tempo_ms:.1f}ms")
62
63     # 5. Executa experimento para cada modelo
64     treinar_e_avaliar(modelA, "Modelo A (Logístico)")
65     treinar_e_avaliar(modelB, "Modelo B (MLP)")

```

Código-fonte 1 - Experimento em Python (Python)

Fonte: Elaborado pelo autor (2026)

Rodando o código evidenciado, obtemos algo como:

```

1  Modelo A (Logístico): Acurácia = 85.00% | Parâmetros = 3 |
Tempo treino = 23.5ms
2  Modelo B (MLP): Acurácia = 96.00% | Parâmetros = 31 | Tempo
treino = 27.4ms

```

Código-fonte 2 – Resultado do experimento em Python

Fonte: Elaborado pelo autor (2026)

Observação: usamos um artifício de temporização com `torch.cuda.Event` para medir o tempo de treino. Para modelos tão pequenos, o tempo absoluto é baixo (na ordem de algumas dezenas de milissegundos) e a diferença entre eles não é significativa nesse exemplo – mas em cenários reais, modelos complexos podem demorar horas para treinar, enquanto modelos simples treinam em minutos.

Analisando os resultados do nosso experimento: o Modelo A (regressão logística) acertou ~85% dos casos de teste, enquanto o Modelo B (rede neural

simples) acertou ~96%. Como esperado, o modelo mais complexo teve desempenho melhor, pois conseguiu capturar a relação não linear dos dados (make_moons produz classes em forma de meia-lua entrelaçadas, que não são separáveis por uma linha). Em termos de complexidade, vemos que o Modelo A possui apenas 3 parâmetros treináveis, contra 31 do Modelo B. O tempo de treinamento, embora quase igual nos dois (cerca de 0,02-0,03 segundos, diferenças irrelevantes aqui), em geral crescerá conforme aumentamos o número de parâmetros e operações. Ou seja, se escalarmos para modelos bem maiores, poderíamos esperar diferenças substanciais de tempo e custo computacional.

O código demonstra uma forma reprodutível e automatizada de comparar modelos. Garantimos equidade: ambos viram exatamente os mesmos dados de treino e teste e começamos com pesos aleatórios fixados (seed = 42) para eliminar variações aleatórias entre as execuções. Além disso, imprimimos métricas de desempenho (acurácia) lado a lado com indicadores de custo (número de parâmetros, tempo de treinamento) – isso é importante em projetos reais, pois frequentemente há um trade-off entre desempenho e eficiência. No nosso caso, o Modelo B claramente traz um ganho considerável de acurácia (~11 pontos percentuais) a um custo computacional pequeno o suficiente para ser desprezível.

Portanto, seria uma escolha justificada optar pelo modelo mais complexo. Em situações reais, essa decisão seria elaborada balanceando também outros critérios, como tempo de inferência (latência ao fazer previsões ao vivo), uso de memória, interpretabilidade e até consumo energético no caso de modelos muito grandes. Vamos agora dissecar os aspectos teóricos e as melhores práticas que cercam projetos de comparação de modelos, apoiando-nos em fundamentos matemáticos e estatísticos para entender por que devemos conduzir esse processo de forma cuidadosa.

SAIBA MAIS

Como vimos, comparar dois (ou mais) modelos envolve muito mais do que rodar ambos e ver quem acerta mais. Nesta seção, vamos aprofundar os fundamentos que garantem uma comparação justa, estável e informativa. Abordaremos formalmente questões como: que condições garantem que um modelo mais complexo traga benefício? como mensurar a incerteza nas diferenças de desempenho? como evitar conclusões precipitadas devido a variações aleatórias? e como planejar experimentos de forma a economizar tempo e recursos?

Equilíbrio entre complexidade do modelo e volume de dados

Uma lição central do aprendizado de máquina é que um modelo precisa ser compatível com a quantidade de informação disponível nos dados. Assim como no escalonamento de pods exigimos que a capacidade atenda à carga, aqui precisamos que o “poder de representatividade” do modelo não exceda em demasia o sinal presente nos dados. Podemos definir uma espécie de razão de utilização de dados: $\rho = \frac{p}{N}$, onde p é o número de parâmetros livres do modelo (ou outra medida de complexidade, como dimensão de VC) e N é o número de exemplos de treinamento. Se ρ for muito maior que 1, ou seja, parâmetros em excesso para poucos dados, o modelo tende a sobreajustar – consegue “memorizar” as amostras de treino, mas falha em generalizar para novas amostras (pega o ruído como se fosse padrão).

Em outras palavras, ele se torna instável fora da amostra de treino, análogo a um sistema de filas onde a carga excede a capacidade e o backlog cresce indefinidamente. A condição ideal de estabilidade é $\rho < 1$, ou melhor, $\rho \ll 1$ para termos margem. Modelos simples (p pequenos) têm viés mais alto (não capturam toda a estrutura, underfitting se $\rho \ll 1$ demais), enquanto modelos complexos (p grande) têm variância mais alta (overfitting se ρ chega a 1 ou ultrapassa). Isso nos remete à conhecida decomposição do erro de generalização:

$$\text{Erro}_{\text{total}} = \text{Bias}^2 + \text{Variância} + \text{Ruído irreducível} (\sigma^2).$$

Modelos simples sofrem com erro de Bias elevado (não importa o quanto de dados tenhamos, há um limite inferior de erro pois a hipótese é limitada), já modelos complexos podem atingir bias muito baixo, mas pagam o preço de variância maior (resultados altamente sensíveis às flutuações dos dados de treino). Em um projeto de comparação, muitas vezes comparamos um modelo “simples” vs um “complexo” sob a ótica desse trade-off. Qual deles terá menor erro total? Depende de quanto dados dispomos e quão complexa é a relação nos dados. Se aumentarmos N , a variância tende a cair (modelos complexos se beneficiam de mais dados, pois σ^2 diminui), enquanto bias é intrínseco à hipótese e não muda com N . Em um regime de muitos dados, um modelo mais complexo geralmente se sobressai, pois o N grande ameniza sua variância e o bias já era baixo de início. Já com poucos dados, modelos complexos costumam decepcionar – o caso clássico de sobreajuste.

Há até regras empíricas: por exemplo, em certos cenários, duplicar o número de amostras reduz pela metade o erro de generalização (quando dominado por variância), sugerindo uma relação aproximadamente $\text{erro} \propto \frac{1}{N}$. Essa “lei de potência” lembra a Lei de Little das filas, que no nosso contexto diria: se você quer reduzir o backlog de erro, tem que prover mais capacidade (mais dados). Claro, isso é um panorama simplificado – na prática nem todo modelo segue exatamente ‘ $1/N$ ’ mas é comum vermos diminuição de erro com o aumento de ‘ N ’ de forma sublinear.

O importante é: ao comparar dois modelos, leve em conta quantos dados você tem. Um modelo mais complexo pode parecer melhor em treino, mas seu ganho em teste pode não se manifestar até que se tenha um volume de dados suficiente para “alimentá-lo”. Portanto, uma dica de projeto é adequar a complexidade do modelo ao tamanho do dataset. Em casos limítrofes (N próximo ou maior que N_{crit}), costuma-se recorrer a técnicas de regularização (que efetivamente diminuem a variância impondo um custo na complexidade) ou optar por modelos mais simples até que mais dados sejam coletados.

Comparação estatisticamente significativa

Quando medimos o desempenho de dois modelos em um conjunto de teste ou via validação cruzada, estamos lidando com estimativas sujeitas a aleatoriedade.

Assim como no HPA um pequeno ruído nas métricas de CPU não deveria acionar mudanças drásticas, aqui não devemos trocar de modelo por diferenças que podem muito bem ser fruto do acaso. Como saber se o modelo B é realmente melhor que o A ou se teve sorte? Para isso, utilizamos ferramentas estatísticas clássicas. Por exemplo, suponha que no teste o modelo A acertou 85% e o B 88%. Essa diferença de 3 pontos percentuais pode não ser significativa se o teste tiver poucas amostras. Uma abordagem é calcular um intervalo de confiança para cada acurácia. Se o modelo A foi testado em n exemplos com acurácia observada $\hat{p}$, o erro padrão da proporção vale $\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$. Para n grande podemos aproximar a distribuição da acurácia por normal, então um intervalo de 95% seria:

$$\hat{p} \pm 1.96 \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}.$$

Se os intervalos de A e B se sobrepõem bastante, não há evidência de que um supera o outro. Outra técnica, especialmente útil quando comparamos modelos nos mesmos dados (como na validação cruzada pareada fold a fold), é usar um teste t pareado ou o teste não-paramétrico de Wilcoxon sobre as diferenças de desempenho em cada partição. Por exemplo, se fizermos uma 5-fold cross validation, vamos obter 5 diferenças $d_i = \text{score}_{B^{(i)}} - \text{score}_{A^{(i)}}$. Podemos calcular a média $\bar{d}$ e o desvio s_d dessas diferenças e formar uma estatística $t = \frac{\bar{d}}{s_d/\sqrt{5}}$ com 4 graus de liberdade para testar $H_0: \bar{d} = 0$. Se $|t|$ ultrapassar o crítico (aprox. 2.78 para 4 df a 95%), rejeitamos H_0 – ou seja, a diferença é estatisticamente significativa.

Em problemas de classificação binária comparando dois classificadores no mesmo conjunto de teste, é comum empregar o Teste de McNemar, que foca nas instâncias em que os modelos discordam. Elaborando ou não esses cálculos manualmente, o fundamental em um projeto é incluir medidas de incerteza nas comparações. Muitos pacotes de AutoML já reportam intervalos ou realizam testes sob o capô. Em contextos críticos (por exemplo, validar um novo modelo para uso em medicina), costuma-se exigir que o novo modelo supere o antigo com folga estatística de 95% ou 99%. Isso evita “oscilações” de modelo para lá e para cá a

cada nova amostra (um análogo das oscilações do autoscaler reagindo a qualquer ruído).

Reprodutibilidade e controle de variáveis

Uma boa prática que apareceu no nosso hands on e merece reforço teórico é a reprodutibilidade. Não adianta comparar dois modelos se um foi treinado com um conjunto de hiperparâmetros ou uma configuração de hardware diferente do outro, a não ser que você contabilize rigorosamente essas diferenças. Idealmente, ao comparar modelos, controle todas as variáveis exceto o modelo em si. Isso significa usar o mesmo conjunto de treino/validação (ou dobras de cross validation) para ambos, usar o mesmo procedimento de pré-processamento nos dados e rodar em condições computacionais semelhantes (por exemplo, se um modelo for treinado em GPU e outro na CPU, verifique se tempos e resultados não são afetados injustamente por isso). Documente versões de biblioteca e seeds aleatórias – isso permite reproduzir a comparação mais tarde ou por outra pessoa.

Hoje existem ferramentas como MLflow e Weights & Biases que automaticamente registram os parâmetros de cada execução, garantindo que possamos repetir o experimento e obter os mesmos resultados. Reprodutibilidade é essencial para confiança: em 2018, um caso notório foi relatado em que um laboratório implementou um novo modelo de detecção de fraudes que parecia excelente em ambiente de teste, mas ao tentar reproduzir o treinamento, os resultados divergiam. Descobriu-se que pequenas variações não controladas (seed diferente e embaralhamento de dados distintos) estavam criando a ilusão de superioridade. Desde então, muitas equipes incorporam nos seus projetos de comparação o hábito de rodar cada experimento várias vezes com inicializações diferentes e tomar a média – se o modelo B for consistentemente melhor na média, temos evidência robusta; se cada execução obtém um vencedor diferente, provavelmente ambos os modelos têm desempenhos equivalentes dentro da margem de variação aleatória.

Controle de custo computacional e eficiência

Em ambientes de produção de ML, recursos computacionais custam dinheiro. “Orçamento de GPU/CPU” não é só figura de linguagem: muitas empresas alocam cotas de horas de GPU para times de Data Science. Por isso, projetos de comparação de modelos precisam equilibrar ambição com viabilidade. Se você tem dez modelos candidatos, mas só tempo (ou dinheiro) para treinar três a fundo, precisa priorizar. Uma estratégia inspirada no “scale-to-zero” do KEDA é usar etapas progressivas: começar treinando todos os candidatos de forma superficial (por exemplo, por poucas épocas ou com um subconjunto pequeno de dados) – isso consome pouco recurso e já pode eliminar modelos claramente inferiores.

É como deixar certos modelos “desligados” até que mostrem algum potencial. Depois, com os dois ou três mais promissores, aí sim investir treino completo com todos os dados e maior afinco (escala de 1 até N replicações, analogamente). Essa abordagem evita gastar horas de GPU em um modelo que desde cedo já dava indícios de baixo desempenho. Muitas práticas de AutoML adotam essa ideia através de algoritmos multi-fidelidade, como Hyperband: eles avaliam parcialmente muitos modelos e param precocemente aqueles com desempenho ruim, concentrando recursos nos potenciais vencedores.

Outra analogia ao autoscaling é que podemos desligar modelos que não são necessários em determinados cenários. Por exemplo, suponha que você tenha um modelo extremamente complexo que vence os outros apenas em dados de altíssima dimensão. Se você tiver um modo de detectar quando esse cenário ocorre, pode ativar esse modelo só nessas horas, mantendo um mais simples nas demais. Esse conceito de usar diferentes modelos conforme gatilhos externos – como características dos dados de entrada ou requisitos de latência do momento – é avançado, mas existe. Alguns sistemas online treinam um conjunto de modelos e têm um orquestrador que, dado o contexto de uma requisição, decide qual modelo usar (priorizando velocidade ou acurácia conforme o caso). Assim, os modelos “dormem” quando não adequados e “acordam” quando um evento condizente ocorre.

Oscilações e histerese na escolha de modelos

Vamos supor que em um mês o Modelo B pareça melhor, no mês seguinte os dados mudam um pouco e o Modelo A assume a liderança, depois volta para B, e assim por diante. Se formos trocando o modelo em produção a cada oscilação leve, incorreremos em instabilidade – os usuários (e desenvolvedores) ficarão confusos com o sistema mudando o tempo todo, possivelmente sem ganhos claros. Isso se assemelha à oscilação de pods que o HPA enfrenta sem histerese: subir e descer constantemente em torno de um limiar. A solução similar aqui é introduzir critérios de decisão com margem de segurança. Uma regra prática muito adotada é a regra do 1 desvio-padrão (1-SE): se um modelo mais simples alcançou resultado dentro de 1 desvio-padrão do resultado do modelo mais complexo, prefira o mais simples. Ou seja, só promova o modelo mais complexo se ele claramente supera o simples por uma margem além da variação esperada.

Essa histerese evita trocas “vai-e-vem” de modelos por ganhos minúsculos. Outra técnica: estabelecer um período mínimo de teste A/B antes de efetivar a mudança – por exemplo, mesmo que offline o modelo B pareça melhor, colocamos ele para 10% dos usuários durante duas semanas; se ao final desse tempo as métricas de negócio melhorarem significativamente, aí sim trocamos 100%. Isso equivale a ter uma janela de estabilização, tal qual o HPA permite configurar para não reduzir pods mais de uma vez a cada tantos segundos.

Do ponto de vista formal, o processo de ajustar modelos pode ser visto como um sistema de controle de realimentação: medimos a performance atual e decidimos um “atuador” (qual modelo usar ou se treinamos um novo) para chegar mais perto da meta (melhor performance possível, respeitando constraints). Se medirmos muito frequentemente e reagirmos agressivamente, corremos risco de overshooting ou undershooting. Overshooting aqui significaria adotar um modelo muito complexo ou ajustar hiperparâmetros exageradamente com base em uma leitura de desempenho que na verdade era circunstancial – resultado: performance piora em vez de melhorar. Undershooting seria demorar demais para se adaptar, deixando o sistema operar aquém do ótimo por inércia. Encontrar o ponto certo implica definir o “ganho” do controlador (quão sensível somos às diferenças de modelos) e considerar o

“atraso” (tempo de treinamento, tempo de coleta de métricas) para evitar instabilidade.

Um estudo clássico mostrou que reagir a métricas de validação em intervalos muito curtos pode levar a sobreajuste das próprias métricas de validação (fenômeno similar ao controlador que amplifica ruído). Por isso, filtros e amortecimento são bem-vindos: olhar médias móveis das últimas iterações em vez do último valor, impor limites de mudança (ex.: “não aumentar a complexidade do modelo mais que 2x de uma vez” – análogo a políticas de HPA que limitam escala para evitar saltos abruptos). A formulação matemática precisa foge do escopo introdutório, mas a intuição é clara – há um laço de feedback onde decisão e medição se influenciam e métodos de teoria de controle (PIDs etc.) podem ser empregados para tornar esse laço estável.

Buffering e atualizações periódicas vs. contínuas:

Em sistemas de software, às vezes aceitamos pequenas degradações temporárias de serviço (como uma fila crescer um pouco) em prol de eficiência global. Em modelos de ML, podemos tolerar um certo “backlog de desempenho” antes de agir. Por exemplo, se um conceito nos dados mudou levemente hoje, talvez não seja crítico retreinar o modelo imediatamente – podemos esperar acumular mais dados sobre essa mudança e só então atualizar o modelo. Isso é semelhante a permitir que a fila de eventos cresça até certo ponto antes de escalar pods.

Essa estratégia de atualização em lote ou programada (por exemplo, retreinar modelos toda semana, em vez de a cada hora) serve para amortecer flutuações de curto prazo e lidar melhor com rajadas de novidades no dado. Você colhe um lote de mudanças e lida com todas de uma vez, otimizando o uso de recursos. Claro, isso traz latência na adaptação: durante aquela semana, o modelo pode estar um pouco defasado. A decisão depende dos SLOs: se a aplicação pode tolerar um leve decréscimo na acurácia por alguns dias, ganhamos em estabilidade e simplicidade operacional ao atualizar menos frequentemente. Já se cada ponto percentual de erro conta a cada momento (ex.: sistema de trading auto-adaptativo), aí precisamos de um pipeline mais responsivo, possivelmente com atualizações contínuas (online learning).

No caso de bursts, pense no lançamento de um produto que gera dados muito diferentes temporariamente – é parecido com um pico de eventos. Em vez de treinar um modelo totalmente novo assim que começa o burst, podemos deixar o modelo atual absorver o impacto (mesmo que o erro suba um pouco), enquanto monitoramos se o padrão persiste. Se for só um outlier de um dia, economizamos o esforço de mudar tudo para depois voltar atrás. Muitas equipes implementam gatilhos com histerese para retreinamento: por exemplo, “retraine o modelo somente se o erro em produção ultrapassar 5% por 3 dias consecutivos” (subiu de 2% para 6%, manteve – claramente um desvio persistente). Aqui há dois limiares: ao exceder 5% consistentemente, ativa a mudança, mas talvez só desativem ou revertam o modelo se cair abaixo de, digamos, 3% de erro por um tempo – prevenindo ficar retrain on/off caso oscile em torno de 4-5%. Esses números são arbitrários, mas ilustram o princípio de banda morta.

Considerações multi-objetivo (SLOs) e além da acurácia

Outro ponto interessante e análogo à evolução do KEDA (que passou a mirar SLOs e custos) é que a meta ao escolher modelos nem sempre é maximizar apenas acurácia. Podemos ter múltiplos objetivos: maximizar desempenho e minimizar latência ou aumentar precisão sujeita a um limite de custo mensal em nuvem etc. Isso leva a formulações como:

$$J(\text{modelo}) = \text{Acurácia} - \lambda \times \text{Custo},$$

onde λ é um peso que converte custo (e.g. segundos extras por previsão ou dólares por hora de GPU) no mesmo “valor” da acurácia. Escolher λ pode ser delicado, mas esse tipo de pontuação ajuda a formalizar o problema de comparação de modelos como uma otimização multicritério. Em um projeto, você pode testar modelos e calcular, por exemplo: Modelo A tem 85% acurácia e custa \$10/dia em processamento, Modelo B 90% mas \$50/dia. Dependendo de λ (quanto vale 5% de acurácia em termos de dinheiro para você), a decisão muda.

Em projetos, é preciso definir claramente seus SLOs: não só acurácia, mas também metas de tempo de resposta, limites de memória, exigências de interpretabilidade etc. Então, a comparação de modelos deve levar em conta todas

essas dimensões. Você pode até montar uma tabela comparativa (um “placar”) com cada modelo e suas métricas em cada critério – muitas vezes, haverá um modelo que domina (melhor ou igual em tudo), mas caso não haja, você terá de pesar o que é mais importante.

Arquiteturas e padrões operacionais em experimentos (MLOps)

Assim como arquiteturas de software separam responsabilidades (um container faz X, outro Y), em experimentos de ML temos padrões estabelecidos para organizar o processo. Um exemplo é o Champion-Challenger (ou “Blue-Green” para modelos): o modelo atualmente em produção é o champion (campeão) e novos modelos treinados são challengers (desafiantes). Em vez de simplesmente substituir o modelo assim que um challenger mostra méritos offline, implementa-se um teste controlado – por exemplo, um shadow deployment: o challenger roda paralelo ao champion em produção, recebendo exatamente as mesmas requisições mas em sombras, sem afetar usuários, e seus resultados são registrados.

Se após um tempo o challenger consistentemente supera o champion nas métricas de negócio (por exemplo, taxa de cliques, receita gerada etc.), então ele “desbanca” o champion. Caso contrário, é descartado. Esse padrão evita surpresas, pois mesmo um modelo com melhor métrica offline pode falhar em algum aspecto não capturado nos dados de teste (a vida real tem peculiaridades!). Portanto, um projeto completo de comparação muitas vezes se estende até a fase de deploy controlado. Grandes empresas (Google, Amazon, Meta) utilizam intensivamente esse método: qualquer novo algoritmo passa por fases de offline A/B, depois online em pequeno tráfego (canary deployment), antes de ser totalmente promovido.

Do lado dos pipelines automatizados, frameworks de MLOps como Kubeflow, MLflow e AWS SageMaker oferecem componentes específicos para experimentação. Por exemplo, há operadores que facilmente executam validação cruzada paralelizada em diversos modelos e consolidam os resultados; existem orquestradores de hiperparâmetros que integram com schedule de recursos (evitando que você exploda suas GPUs rodando 100 experimentos simultaneamente sem querer).

Em termos de padrão operacional, recomenda-se em projetos definir antecipadamente: (1) quais modelos serão comparados e com quais hiperparâmetros (planejamento experimental), (2) qual será a metodologia de avaliação (por exemplo, “usar 20% dos dados como teste final e não tocar nesses durante as comparações, usar 5-fold CV nos 80% de treino para estimar performance de cada modelo”), (3) critérios de seleção (e.g. “modelo escolhido deve ter acurácia ≥ 2 pontos acima do atual e tempo de inferência $\leq 0.5s$ ”) e (4) como será feita a implantação e o monitoramento do novo modelo (talvez com um rollback preparado caso o novo piore algo inesperado).

Note que isso envolve tanto aspectos técnicos quanto organizacionais – por isso o termo “padrões operacionais”. Equipes maduras chegam a ter documentos de governança dizendo: “Não promover modelo sem teste estatístico significativo”, “Registrar todos os experimentos no repositório X”, “Se modelo novo falhar em produção, reverter imediatamente ao anterior que está versionado” etc.

Tendências: AutoML, meta-learning e personalização de métricas

No horizonte, observarmos duas grandes linhas evolutivas. Primeiro, a automação ainda maior do processo de comparação – o chamado AutoML. Já mencionamos como ele tenta modelos diversos automaticamente. Algumas plataformas de AutoML agora fazem otimização multiobjetivo (por exemplo, o Google Vizier permite definir múltiplas métricas alvo). Isso significa que em um futuro próximo, projetos de comparação poderão consistir basicamente em definir o que otimizar (métricas e restrições) e deixar a ferramenta explorar o como (testando modelos, arquiteturas, features). Contudo, mesmo nesses cenários, o papel humano continua crucial na interpretação e validação final – lembrando da Netflix, até um AutoML poderia cuspir um ensemble complexo vencedor em RMSE, mas alguém tem que julgar se vale a pena usá-lo.

A segunda tendência é integrar novas preocupações nas métricas. Além de custo e SLOs já mencionados, há muita discussão sobre fairness (justiça) e robustez. Um projeto moderno de comparação talvez avalie não só acurácia global, mas também acurácia por subgrupo (ex.: por gênero ou etnia, em aplicações sensíveis) e prefira modelos que tenham performance mais equitativa. Já existem

toolkits, como o AI Fairness 360 da IBM, que podem gerar métricas de viés – e essas podem se tornar “gatilhos externos” do nosso processo, determinando escolhas de modelos não apenas pelo desempenho médio, mas também pelo impacto ético. Do lado de robustez, pode-se comparar modelos pelo quão bem resistem a entradas fora da distribuição ou ataques adversariais. Esses critérios adicionais aumentam a complexidade da comparação, mas refletem demandas reais da sociedade e do mercado.

Resumindo, vimos que um projeto de comparação de modelos de ML envolve ciência e engenharia lado a lado: estatística para garantir conclusões confiáveis, computação para viabilizar experimentos sem extrapolar recursos e alinhamento com objetivos práticos para tomar decisões acertadas. Tratar esse processo com a mesma seriedade que tratamos, por exemplo, o autoscaling de um sistema, é o que diferencia uma solução de IA experimental de um produto de IA confiável e otimizado.

MERCADO, CASES E TENDÊNCIAS

Vamos agora explorar alguns casos reais (ou baseados em situações reais) e tendências de mercado relacionados a comparação de modelos para ver como esses conceitos se aplicam fora do papel. Você vai perceber que as maiores lições vêm tanto de sucessos quanto de fracassos na hora de escolher modelos.

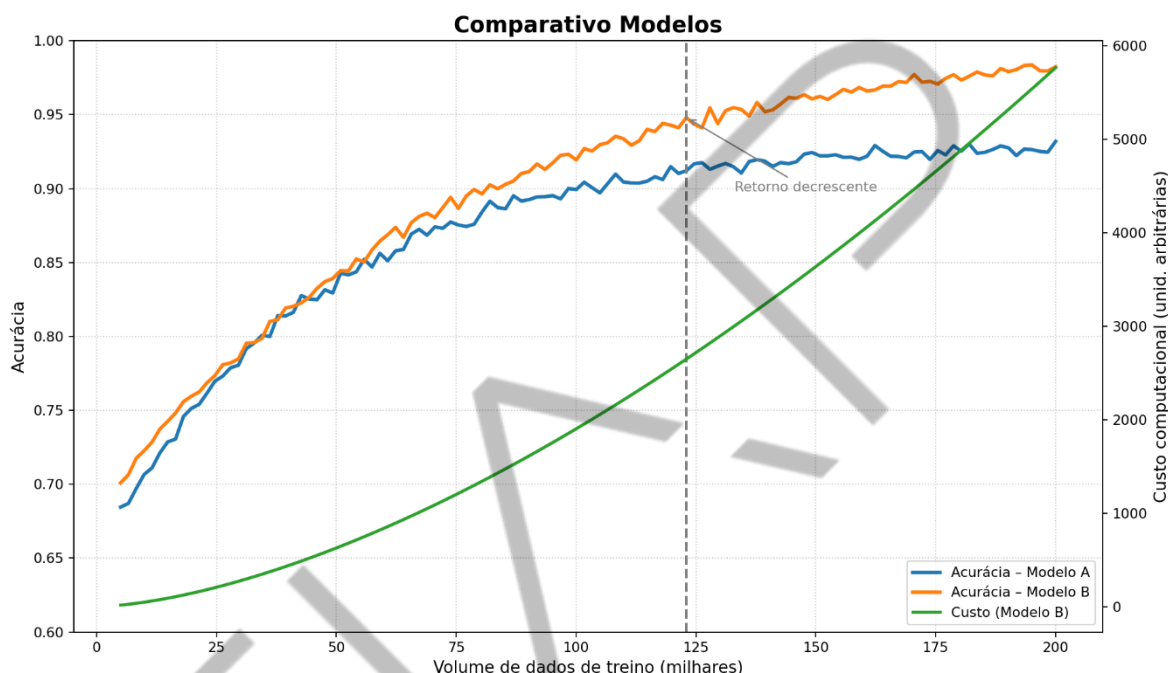


Figura 1 - Comparativo Modelos
Fonte Google Imagens (2026)

No gráfico ilustrado na figura 1, podemos imaginar o seguinte cenário: o Modelo B (mais complexo) eventualmente ultrapassa o Modelo A (mais simples) em acurácia conforme aumentamos o conjunto de treino – porém, seu custo (verde) também cresce bem mais rápido. Situações assim são comuns: para certos tamanhos de dataset, um modelo simples pode ser “bom o suficiente” comparado a outro muito pesado. Empresas lidam com esse tipo de análise para decidir se vale a pena coletar mais dados e treinar um modelo maior ou se o retorno não compensa.

“Até onde compensa ir?” é uma pergunta-chave. Esse tipo de gráfico de learning curve versus cost curve ajuda a responder. Por exemplo, se dobrar o dataset e usar um modelo mais sofisticado vai custar o dobro em infraestrutura, mas melhorar a acurácia só de 90% para 92%, talvez a decisão de negócio seja ficar com a opção mais barata – a não ser que aqueles 2% sejam críticos (como em saúde).

Esse trade-off desempenho-custo, outrora negligenciado, hoje está no centro de muitas decisões de IA nas empresas.

O caso real da competição Netflix Prize em 2009 ensina que ganhos algorítmicos marginais podem não se justificar do ponto de vista prático. Não significa que competições não valham: o Netflix Prize gerou muitas ideias inovadoras (ensembles, modelos de fatoração de matrizes aprimorados etc.), que a Netflix aproveitou de maneiras indiretas. Mas a lição de ouro é alinhar comparações de modelos com métricas de negócio. Hoje, antes de embarcar em um projeto de melhoria de modelo, times se perguntam: qual impacto esse 1% de acurácia terá na métrica final (vendas, cliques, engajamento)? Vale o investimento?

Falando em competições, a cultura do Kaggle e demais plataformas popularizou bastante as técnicas de comparação rigorosa. Os melhores competidores sabem que para vencer precisam espremer até o último décimo de ponto – então usam validação cruzada extensiva, ensembles e model stacking e monitoram cuidadosamente se um novo modelo realmente adiciona ganho (usando métodos estatísticos para não se enganar com variação aleatória). Isso permeou o mercado: hoje muitas empresas, ao contratar cientistas de dados, esperam que eles(as) adotem uma mentalidade “kaggler” para avaliar modelos internamente. Por outro lado, competições também ensinam o outro lado da moeda: o modelo campeão muitas vezes é um Frankenstein difícil de reproduzir e implantar.

Assim, nas empresas há um constante balanço entre “sim, vamos comparar todos esses modelos e até combinar vários se precisar” e “calma, isso precisa rodar em produção 24/7”. Por isso, vemos muitas vezes soluções híbridas: durante a fase de pesquisa usa-se de tudo (ensembles gigantes etc.), mas ao entregar para produção simplifica-se a pipeline (por exemplo, destilando o ensemble em um único modelo próximo em desempenho). Essa transição às vezes falha: há casos de empresas que aderiram cegamente ao modelo vencedor de um hackathon interno sem notar que era inviável manter aquilo rodando – e acabaram tendo que voltar atrás.

Um caso fictício ilustrativo: a startup X desenvolveu um modelo de IA para triagem de currículos. Em testes internos, um modelo complexo (uma rede profunda) teve 2% melhor acurácia que um modelo de árvore de decisão. Empolgados, adotaram a rede e a integraram no sistema de RH. O que não consideraram: o

modelo profundo demorava 5 segundos para processar cada currículo, contra 0.1s da árvore, e eles recebiam milhares de currículos por hora. O sistema de triagem ficou lento e caro em produção. Além disso, meses depois um auditor externo pediu explicações sobre as decisões do modelo (para verificar vieses) e a rede neural era praticamente uma caixa-preta, ao contrário da árvore que era bem mais interpretável. Resultado: a startup teve que voltar correndo para o modelo antigo, depois de já ter investido tempo integrando a rede complexa.

Moral da história: comparar modelos envolve múltiplos critérios – aqui, ignoraram latência e a interpretabilidade no processo decisório. Hoje, muitas empresas evitam esse erro classificando modelos em camadas: se uma decisão precisa ser explicável por compliance, já descartam opções do tipo “caixa-preta” no comparativo ou então planejam métodos de interpretabilidade paralelos. Se a aplicação é tempo-crítica (como um sistema antifraude online), dão peso altíssimo à métrica de latência nas comparações. Assim, evitam surpresas.

Nos casos positivos, podemos citar, por exemplo, o Facebook. O Facebook/Meta tem um sistema robusto de experimentação chamado FBLearner Flow, onde podem rodar milhares de experimentos de modelo em paralelo. Eles relataram ganhos enormes em elementos como previsão de cliques em anúncios ao conseguir treinar e comparar rapidamente variações de seus modelos e features. Mas internamente sempre há a preocupação de custo: cada experimento de modelo consome centenas de GPUs, então eles implementaram job schedulers inteligentes para enfileirar experimentos e pausar aqueles de baixa prioridade se os recursos estiverem escassos. Isso é praticamente aplicar um autoscaling/queue de jobs de ML.

Resultado: é possível manter um alto throughput de experimentos (explorando muitas ideias de modelos) sem estourar o orçamento de nuvem. Esse é um padrão em empresas grandes – veja a infraestrutura de empresas como Google (com o sistema Vizier para otimização Bayesian de hiperparâmetros), Microsoft (com Azure AutoML e seu orchestrador), Uber (com Michelangelo) etc. Todas investiram não só em criar modelos melhores, mas em sistemas para comparar modelos melhores. Isso tornou o processo mais democrático (engenheiros de menor experiência podem rodar AutoML e obter bons baselines) e muito mais rápido. Alguns relatam que

tarefas que levavam semanas de tentativa e erro manual agora são resolvidas em dias com automação.

Comparativo Modelos

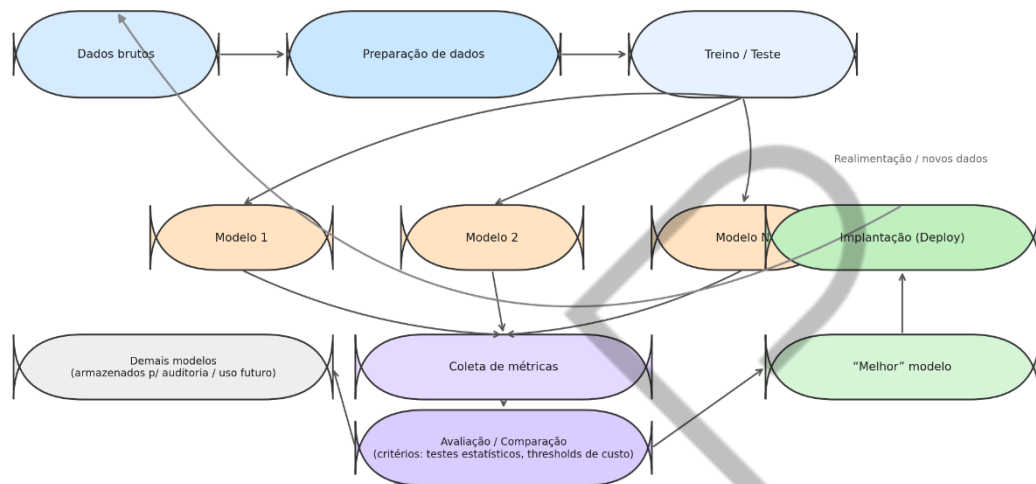


Figura 2 - Pipeline de Modelos
Fonte Google Imagens (2026)

O diagrama da figura 2 sintetiza como empresas estruturam o processo. Note que há um módulo de avaliação central – isso evita vieses: todos os modelos são avaliados com o mesmo código, mesmas métricas. Muitas falhas ocorriam quando cada pesquisador testava seu modelo “do seu jeito”; hoje padroniza-se para que a comparação seja justa. Outra coisa a destacar: o modelo “perdedor” não é simplesmente jogado fora; ele pode ficar guardado. Por quê? Reprodutibilidade (saber que aquele modelo foi testado e não funcionou) e até aprendizado organizacional – pode ser que no futuro, com mais dados, valha a pena retestar aquele modelo; então, se já temos ele versionado e documentado, economizamos esforço.

No horizonte de tendências, duas palavras: AutoML e MLOps (já mencionadas). A convergência delas aponta para um futuro de comparação de modelos contínua e automatizada. Empresas caminhando para IA madura querem que, sempre que novos dados cheguem, o sistema possa por si só treinar alguns modelos candidatos e talvez até realizar um deploy canário do melhor, somente alertando a equipe caso algo saia dos padrões. Ainda estamos alguns passos desse

nível de autonomia total (especialmente pelo risco envolvido), mas já vemos progressos. Por exemplo, o Gmail do Google possuía um modelo de filtragem de spam que era atualizado automaticamente via um pipeline de aprendizado contínuo; só se a performance caísse abaixo de um patamar que o sistema não conseguia recuperar é que humanos intervinham. Esse é um caso de auto-tuning de modelo em produção.

Outra tendência é a especialização de modelos: e se em vez de ter um modelo universal, você tivesse vários modelos especializados e um meta-modelo que escolhe qual usar por contexto (parecido com o que comentamos de gatilhos)? Isso já acontece em pequenos casos, mas pode se tornar mais formalizado. Por exemplo, uma fintech pode ter um modelo de previsão de inadimplência geral, mas descobre que para clientes de certo perfil (digamos, autônomos) outro modelo funciona melhor; em vez de se comprometer com um só, ela mantém ambos e cria uma lógica de alto nível: “se cliente autônomo, use modelo X, caso contrário modelo Y”.

Nesse projeto, a comparação de modelos se dá dentro de subgrupos. O desafio passa a ser comparar não só modelos, mas estratégias de combinação de modelos. Conforme a área evolui, em vez de perguntar “qual modelo é melhor?”, perguntaremos “qual conjunto de modelos e em que condições cada um é melhor?”. Isso lembra ensembles, mas pensados de forma determinística, com base em regras de negócio, não apenas combinando outputs.

Por fim, mencionemos a importância crescente da responsabilidade e auditabilidade. Em setores como financeiro e médico, órgãos reguladores esperam ver relatórios de modelos. Ou seja, se você escolheu o modelo B para conceder crédito, você deve documentar por que B e não A (talvez A discriminava determinadas regiões, ou B tinha menos falso-negativos de fraude). Portanto, os projetos de comparação agora geram não só um “vencedor”, mas todo um dossiê do processo: métricas, gráficos, testes realizados, considerações de custo. Algumas empresas adotam o formato de “registro de decisões” – um documento anexo a cada modelo em produção que resume os experimentos comparativos que o justificaram. Isso é uma tendência saudável de governança, tornando o desenvolvimento de modelos mais parecido com o de software tradicional, onde qualquer mudança em produção passa por revisão e necessita justificativa.

Em suma, o mercado de AI caminha para tornar a comparação de modelos uma atividade contínua, automatizada e multifacetada. Ferramentas assumem grande parte do trabalho braçal (treinar e avaliar dezenas de modelos), mas cabe aos profissionais definirem objetivos claros, checar pressupostos e interpretar resultados. Quem dominar tanto os fundamentos (como fizemos nesta aula) quanto essas práticas consegue navegar nesse espaço de forma eficaz – entregando modelos melhores de forma mais rápida e com justificativa sólida.



O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos em detalhes como conduzir um projeto de comparação de modelos de Machine Learning, desde a motivação até a implantação. Você aprendeu que avaliar modelos de forma rigorosa é essencial para garantir que estamos escolhendo a solução certa pelos motivos certos – não apenas porque pareceu “boa no treino”. Na introdução, vimos as limitações de se basear unicamente em uma métrica estática ou em um único experimento: assim como sistemas reativos simples (HPA tradicional) podem falhar diante de cenários complexos, comparações simplórias de modelos podem nos levar a conclusões erradas.

No hands on, colocamos isso em prática com um exemplo concreto em Python: construímos em poucas linhas um experimento reproduzível para comparar dois algoritmos em um dataset sintético. Pudemos ver na saída como o modelo mais complexo de fato atingiu acurácia maior, porém ao custo de mais parâmetros e (nesse caso mínimo) mais tempo de treino. Essa demonstração refletiu etapas que você aplicaria em um projeto real: preparação de dados conjunta, treinamento sob condições equivalentes, medição padronizada e análise crítica dos resultados.

Em seguida, cobrimos fundamentos teóricos: discutimos o balanço entre complexidade do modelo e volume de dados; revisitamos a decomposição bias-variância-ruído para entender porque modelos mais complexos nem sempre vencem – tudo depende do regime de dados; introduzimos formalmente ferramentas estatísticas para comparação; abordamos reprodutibilidade; e exploramos também a dimensão de custo computacional. Além disso, delineamos o problema de oscilações, formalizamos esse cenário com referências a sistemas de controle, para reforçar que existe uma lógica de retroalimentação na seleção de modelos, e discutimos objetivos múltiplos: incorporar custo, tempo e outros critérios (como fairness) no processo de comparação, possivelmente através de funções objetivo combinadas (ex.: $\text{acurácia} - \lambda \cdot \text{custo}$). Isso trouxe um olhar além da acurácia, alinhado com o que empresas buscam – modelos que sejam não só bons em previsão, mas também eficientes, éticos e alinhados a metas de negócio.

Você deve sair desta aula com uma visão bem fundamentada de como planejar e executar comparações de modelos de ML. Em resumo, vimos por que não podemos confiar em uma única medição ingênua (podendo ser sorte ou enviesada), como utilizar técnicas estatísticas e computacionais para obter avaliações confiáveis (intervalos, testes, re-execuções, controlando aleatoriedade), quando considerar aspectos de custo e outros critérios (sempre! já no design do experimento) e o que fazer com os resultados (tomar decisões informadas, implementar o melhor modelo com confiança, mas monitorando continuamente seu desempenho). Com essas ferramentas, você está apto(a) a conduzir projetos de comparação de modelos de forma profissional, garantindo que a escolha final de um modelo não seja um chute, mas sim o produto de um processo científico e alinhado aos objetivos da aplicação. Boas comparações e bons modelos!

REFERÊNCIAS

BISHOP, C. M. **Pattern Recognition and Machine Learning**. New York: Springer, 2006.

BROWNLEE, J. **Statistical Significance Tests for Comparing Machine Learning Algorithms**. 2020. Disponível em: <https://machinelearningmastery.com/statistical-significance-tests-for-comparing-machine-learning-algorithms/>. Acesso em: 23 fev. 2026.

DEMŠAR, J. Statistical Comparisons of Classifiers over Multiple Data Sets. **Journal of Machine Learning Research**, v. 7, n. 1, p. 1–30, 2006. Disponível em: <https://www.jmlr.org/papers/volume7/demsar06a/demsar06a.pdf>. Acesso em: 23 fev. 2026.

DIETTERICH, T. G. Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. **Neural Computation**, v. 10, n. 7, p. 1895–1923, 1998. Disponível em: <https://doi.org/10.1162/089976698300017197>. Acesso em: 23 fev. 2026.

GÉRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow**. 2. ed. Sebastopol: O'Reilly Media, 2019.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. 2016. Disponível em: <https://www.deeplearningbook.org/>. Acesso em: 23 fev. 2026.

HESTNESS, J.; NARANG, S.; et al. **Deep Learning Scaling is Predictable, Empirically**. 2017. Disponível em: <https://arxiv.org/abs/1712.00409>. Acesso em: 23 fev. 2026.

IBM RESEARCH. **AI Fairness 360: An Open Source Toolkit**. 2018. Disponível em: <https://ibm.biz/AI-Fairness-360>. Acesso em: 23 fev. 2026.

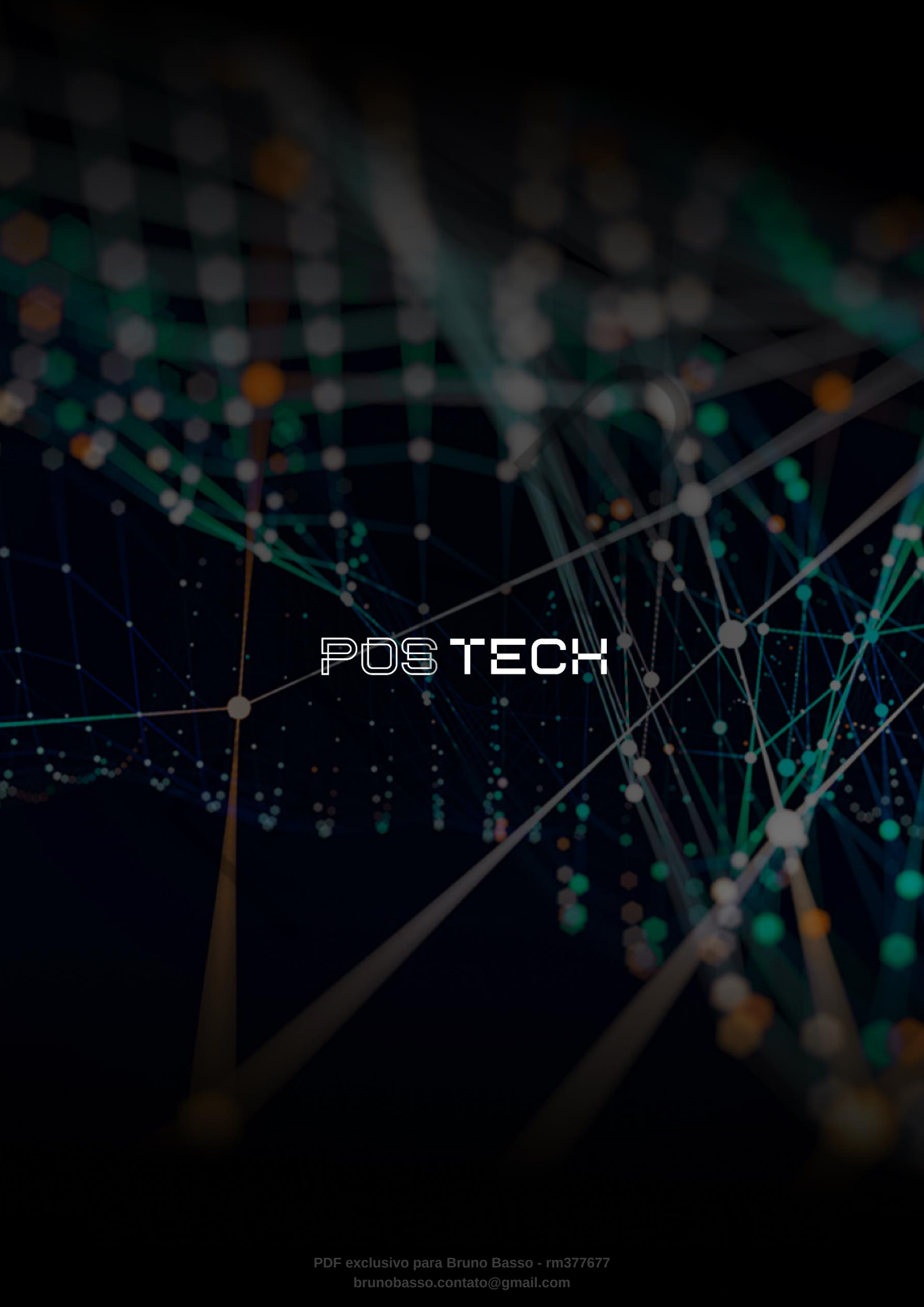
RASCHKA, S. **Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning**. 2018. Disponível em: <https://arxiv.org/abs/1811.12808>. Acesso em: 23 fev. 2026.

YAU, N. **Why the Netflix Prize Never Materialized**. 2012. Disponível em: <https://flowingdata.com/2012/04/17/why-the-netflix-prize-never-materialized/>. Acesso em: 23 fev. 2026.

PALAVRAS-CHAVE

Modelos. Reprodutibilidade. MLOps.

EMENDAS



POSTECH